

INDEX BUILDER

Experimenting indexing large datasets of articles into data structures



Authors: Abigail Siy, Udit Shah, Ananya Shah

PA01 - Index Builder | DS4300 with Professor Fontenot

Name (As in Gradescope)	NEU Email
Ananya Shah	shah.anan@northeastern.edu
Udit Shah	shah.udit@northeastern.edu
Abigail Lane Siy	siy.ab@northeastern.edu

INTRODUCTION

Tasked with identifying a fast way to parse through a large corpus of data which would ultimately be a part of a specialized search engine, the core mission of this project was to evaluate various in-memory indexing data structures for fast and efficient document retrieval. We were working with an (extremely) large corpus of financial news articles that threatened to destroy our PC's RAMs. But, as diligent co-ops, we persisted in our commitment to deliver results to our researcher.

To index this dataset we extracted relevant tokens and metadata from the articles, then built and evaluated the inverted indexes. These indexes were implemented using data structures such as Binary Search Trees (BST), AVL trees, Hash Tables, and Tries. Through rigorous testing with varying index sizes and search patterns (both single-word and multi-word queries), we analyzed the performance characteristics of each data structure.

Each data structure presented its own set of trade-offs: BSTs offered a balanced approach to searching but became unbalanced faster than a tech startup's work-life balance, AVL trees maintained their composure through self-balancing but spent much time reorganizing, and so on.

This project also simulated fundamental aspects of a Relational Database Management System (RDBMS), particularly focusing on how different indexing strategies affect user-driven query performance. Just as an RDBMS uses indexes to avoid full table scans and optimize query execution, our system implements and compares various data structures. And, our experimentation methodology mirrors the real-world challenges database administrators face when optimizing query performance.

DATA STRUCTURE IMPLEMENTATIONS

We used four indexing data structures: a Binary Search Tree (BST), an AVL Tree, a Hash Table (HashMap), and a Trie (Prefix Tree).

Binary Search Tree (BST)

Our BST implementation initially started with a basic recursive approach but evolved significantly due to stack overflow issues with large datasets. So we made some changes:

- Converted all-recursive operations (insert, search, traversal) to iterative implementations using stack-based approaches
- Optimized document ID storage using Sets at each node for $O(1)$ lookup times
- Implemented a non-recursive iterator for efficient tree traversal

After many tries, we were able to create a memory-efficient design that avoided recursion depth limitations.

AVL Tree

Building upon the BST foundation, our AVL implementation adds self-balancing capabilities while maintaining iterative operations. We kept on running into memory issues while indexing the AVL due to the rebalancing processing time and effort. So, during insertion, we maintain a path stack to efficiently update heights and perform necessary rotations without recursive calls. This balancing, while introducing some overhead, ensures consistent $O(\log n)$ operations even with skewed data. A unique aspect of our AVL implementation is the efficient handling of document ID sets during rotations, ensuring no performance degradation during tree reorganization.

Trie (Prefix Tree)

Our Trie implementation is specially optimized for string operations and prefix-based searches. Unique features include:

- Cached set of all words (`_all_words`) to avoid expensive tree traversals
- Dictionary-based child node storage for fast character lookups
- Set-based document reference storage at terminal nodes
- Efficient iterator implementation for term retrieval

The word caching mechanism trades memory for performance, significantly improving common search operations in our financial corpus.

Hash Table

Our HashMap implementation includes collision handling and dynamic sizing. Key optimizations include:

- Chaining-based collision resolution, chosen for its superior handling of string keys
- Dynamic resizing with a load factor threshold of 0.75
- Polynomial rolling hash function for improved key distribution
- Set-based document reference storage for $O(1)$ lookups
- Efficient rehashing during resize operations

The choice of chaining over open addressing was particularly effective for our financial terms, where certain keywords appeared frequently across documents.

Across all implementations, we maintained consistent patterns for document ID storage using Sets, efficient iteration methods, and robust size tracking for performance monitoring. Each structure handles edge cases, from invalid inputs to empty searches, ensuring reliability with our substantial dataset. These implementations evolved through rigorous testing with the large dataset, with particular attention to memory efficiency, recursion limits, and search performance.

EXPERIMENT METHODOLOGY

Our dataset consists of over 30,000 financial news articles in JSON format. Each article contains structured fields including:

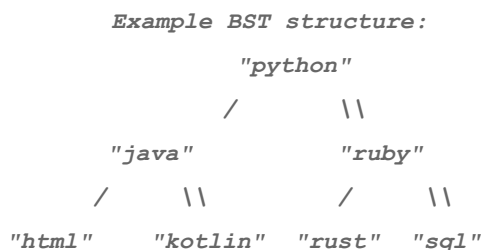
- Title
- Domain (URL)
- Author (if present)
- Preprocessed text content

During preprocessing, we extracted these components:

- Title: Split into terms, preserve original case
- URL: Extract full domain using urlparse
- Author: Store complete author-name
- Preprocessed text: Split into terms, preserve original case
- Document IDs

The processed data was then used to build inverted indexes that mapped tokens to document references, along with labels indicating where each token was found (e.g., "title" or "text").

Here's an example of the data was indexed to BST:



Experimental Setup

To ensure thorough evaluation, we generated search datasets with four distinct components:

1. Component A (Token Queries):

- Random sample of n individual tokens from the index
- Represents single-word search queries
- n is at least 4000 and a multiple of 4 (but we used 3000 and multiple of 4)

2. Component B (Existing Phrase Queries):

- $n/4$ phrases created from Component A tokens

- Each phrase contains 2-3 tokens
- Simulates multi-word search queries

3. Component C (Random Strings):

- n randomly generated strings
- Designed to not exist in the index
- Tests system behavior for unsuccessful searches

4. Component D (Random Phrase Queries):

- n/4 phrases created from Component C strings
- Each phrase contains 2-3 random strings
- Further tests system handling of unsuccessful searches

All components are combined and shuffled to eliminate ordering bias before testing.

We recorded metadata for each experiment run:

- Unique run identifier
- Hardware information (processor type, memory size)
- Index characteristics (type: BST, AVL, Hash Table, or Trie)
- Dataset statistics (number of documents, unique tokens)
- Search dataset size
- Performance metrics (average search time in nanoseconds)

This methodology allowed us to rigorously evaluate the performance characteristics of different index implementations under varying conditions while maintaining experimental consistency and reproducibility.

However, we ran into some problems while running the experiment on our large dataset. While we were able to index all structures, while running the experiments, our laptops faced memory issues. Each run began consuming up to 35GB when it reached AVL Indexes.

So we decided to reduce the dataset size during the experiment run. We ran the experiment for articles found in a single month and reduced the dataset size by 50% and 70%. We then tested it with different base sizes. Initially, we started with 1000 as the base size, increasing by 1000 each, reaching a max of 8000 base size for component generation. We ended up with four results - testing on January2018 (50% of this dataset) with min base size of 1000, April2018 (50% of this dataset) with min base size of 1000, April2018 (70% of this dataset) with min base size of 1000, April2018 (70% of this dataset) with min base size of 3000. We then compared performance across each of these results as well to give a more comprehensive overview.

RESULTS

Index Characteristics

While running the indexing, we created a method `generate_index_statistics()` within our `assign_01.py` which prints the basic characteristics of each index. Our results showcased that all four structures successfully indexed the full dataset. Here are some key observations (for 70% of the April2018 dataset):

Each index contains:

- 128,697 unique terms
- 153,356 total documents

All structures show consistent statistics, indicating proper indexing:

- Term frequency range: 1 to 30,083
- Mean occurrences: 43.22
- Median occurrences: 2.00

Most frequent terms found were reasonable for financial news:

1. "april": 30,083 occurrences
2. "reuters": 26,398 occurrences
3. "company": 23,389 occurrences

This evidence demonstrates that all four data structures were properly constructed and maintained their structural properties while handling the full dataset. The consistent statistics across all structures further validate the integrity of the implementations.

Image 1: Snapshot of Index Characteristics

```
=== INDEX STATISTICS ===
2025-02-06 22:49:52,786 [INFO]
Analyzing BST index...
2025-02-06 22:49:54,337 [INFO] Total Terms: 128,697
2025-02-06 22:49:54,337 [INFO] Total Documents: 153,356
2025-02-06 22:49:54,337 [INFO] Unique Terms: 128,697
2025-02-06 22:49:54,337 [INFO]
Term Frequency Statistics:
2025-02-06 22:49:54,338 [INFO] Min: 1
2025-02-06 22:49:54,339 [INFO] Max: 30,083
2025-02-06 22:49:54,350 [INFO] Mean: 43.22
2025-02-06 22:49:54,358 [INFO] Median: 2.00
2025-02-06 22:49:54,358 [INFO]
Document Type Distribution:
2025-02-06 22:49:54,358 [INFO] title: 428,045
2025-02-06 22:49:54,358 [INFO] text: 5,065,543
2025-02-06 22:49:54,358 [INFO] author: 26,127
2025-02-06 22:49:54,358 [INFO] domain: 42,410
2025-02-06 22:49:54,358 [INFO]
Top 10 Most Frequent Terms:
2025-02-06 22:49:54,394 [INFO] 'april': 30,083 occurrences
2025-02-06 22:49:54,394 [INFO] 'reuters': 26,398 occurrences
2025-02-06 22:49:54,394 [INFO] 'company': 23,389 occurrences
2025-02-06 22:49:54,394 [INFO] 'www.reuters.com': 22,062 occurrences
2025-02-06 22:49:54,394 [INFO] 'said': 19,495 occurrences
2025-02-06 22:49:54,394 [INFO] 'source': 19,110 occurrences
2025-02-06 22:49:54,394 [INFO] 'year': 17,116 occurrences
2025-02-06 22:49:54,394 [INFO] 'new': 15,648 occurrences
2025-02-06 22:49:54,394 [INFO] 'reporting': 15,324 occurrences
2025-02-06 22:49:54,394 [INFO] 'editing': 14,063 occurrences
2025-02-06 22:49:54,394 [INFO]
```

Experimental Results

Average Search Time by Index Type Across Datasets

When we ran a comparison of search time across the different experiment results, we found that the largest dataset seemed to be most efficiently performing across all indexes, especially for the Binary Search Tree. This seems counterintuitive that BST performs better with a larger dataset, but it can be explained by better memory locality and CPU cache utilization with the larger dataset - when more data is consistently accessed, the CPU can better predict and prefetch data, potentially offsetting the expected $O(\log n)$ slowdown.

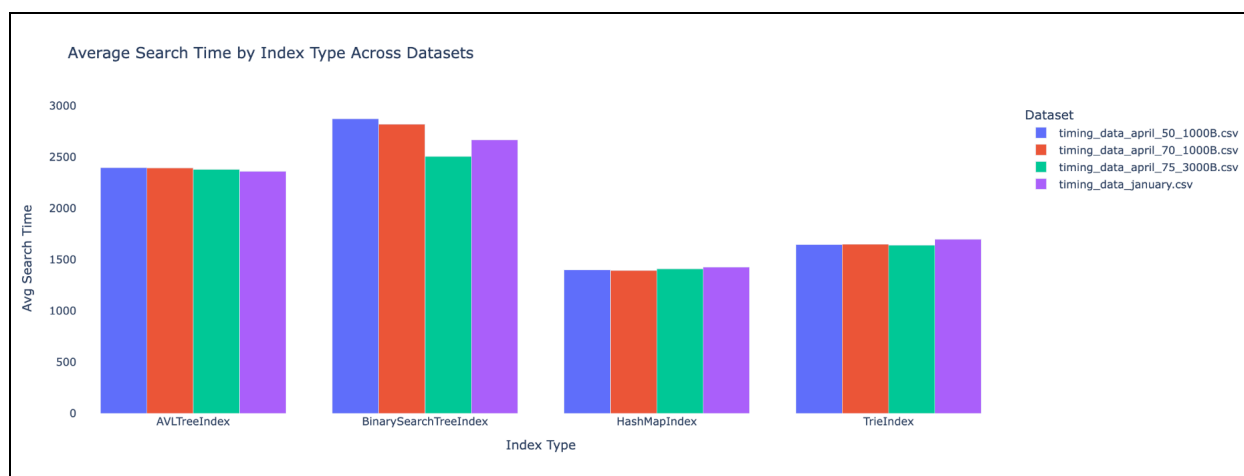


Figure 1: Compares the average search time of the different index types across all the datasets. The Hash map has the lowest average search time among the four index types.

Scalability Analysis

HashMapIndex consistently demonstrates superior performance with the lowest search times (1300-1600ns), followed by TrieIndex maintaining stable performance around 1600-1700ns, while both tree-based structures (AVL and BST) show higher search times ranging from 2200-2800ns. Notably, the BinarySearchTreeIndex displays an unexpected trend where its performance improves with larger datasets, potentially due to system-level optimizations or caching effects. The relatively flat lines for HashMap and Trie implementations indicate excellent scalability across all dataset sizes, aligning with HashMap's theoretical $O(1)$ average search time complexity. The overall hierarchy of performance remains consistent, with hash-based indexing proving the most efficient for pure search operations in this context.

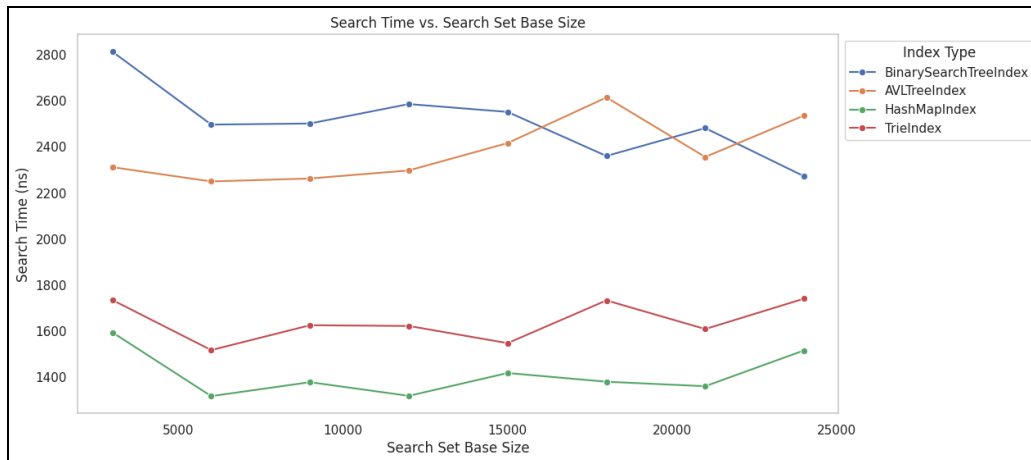


Figure 2: Comparison of the average search times for each data structure

Performance Comparison

As the search set base size increases, the search time also increases, which is more prominent in tree-based indexes like AVL and Binary Search Trees, indicating that their performance declines with larger datasets. Hash Maps exhibit the lowest search times across all base sizes. Trie Indexes perform better than tree-based structures but still take longer than Hash Maps. These results showcase how different indexing structures scale with increasing dataset sizes, highlighting the trade-offs between search efficiency and the complexity of the index structure

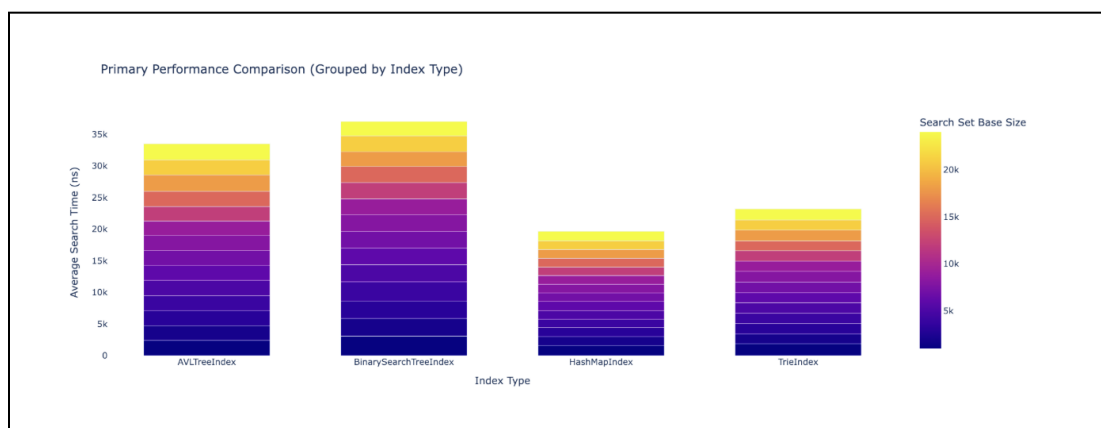


Figure 3: Primary performance comparison of search set base sizes by index type

DISCUSSION

Among HashMap, Trie, AVL, and BST, the HashMap consistently delivered the fastest average search times (1300–1600ns) and showed minimal performance degradation at

scale. The Trie stood out for string-related operations at around 1600–1700ns but demanded more memory. Meanwhile, AVL and BST trailed behind, with AVL balancing predictably (2200–2500ns) and BST showing the widest range (2400–2800ns) due to its sensitivity to data order.

In smaller datasets (fewer than 12,000 terms), differences among the four structures were less pronounced. As data sizes increased, HashMap dominated, while Trie maintained second place. AVL's self-balancing added overhead but ensured steadier performance than an unbalanced BST, which occasionally improved under favorable distributions yet remained less reliable overall.

After looking at these results, we can conclude that for speed and reliability across diverse datasets, HashMap is the strongest choice. Trie only excels where prefix searches or string operations are frequent, offsetting its higher memory cost. **So, we would recommend to the researcher to adopt HashMaps for their specialized search engine.**

CONCLUSION

A limitation was memory consumption, especially for AVL Trees and Tries as it required significant resources for rebalancing and the hierarchical storage. Additionally, the dataset reduction strategy we incorporated in order to avoid memory crashes may have impacted real-world scalability assessments. Hardware constraints also may limit our ability to test larger dataset sizes. Thus, we couldn't conduct a full analysis of long-term efficiency trends.

Despite the limitations we faced, our results provide clear recommendations: HashMap should be used for high-speed lookups, Tries for structured prefix searches, and AVL Trees when there is a necessity for balanced tree structure.